

Why Plurals for Tables?

Because it sounds good in conversation. Really. “Select a Product from products.” And “Order has_many :line_items.”

The intent is to bridge programming and conversation by creating a domain language that can be shared by both. Having such a language means cutting down on the mental translation that otherwise confuses the discussion of a *product description* with the client when it’s really implemented as *merchandise body*. These communications gaps are bound to lead to errors.

Rails sweetens the deal by giving you most of the configuration for free if you follow the standard conventions. Developers are thus rewarded for doing the right thing, so it’s less about giving up “your ways” and more about getting productivity for free.

Rails does this using a simple naming convention. If an incoming request has a controller named (say) `admin/book`, Rails will look for the controller called `book_controller` in the directory `app/controllers/admin`. That is, the final part of the controller name will always resolve to a file called *name*`_controller.rb`, and any leading path information will be used to navigate through subdirectories, starting in the `app/controllers` directory.

Imagine that our program has two such groups of controllers (say, `admin/`*xxx* and `content/`*xxx*) and that both groups define a book controller. There’d be a file called `book_controller.rb` in both the `admin` and `content` subdirectories of `app/controllers`. Both of these controller files would define a class named `BookController`. If Rails took no further steps, these two classes would clash.

To deal with this, Rails assumes that controllers in subdirectories of the directory `app/controllers` are in Ruby modules named after the subdirectory. Thus, the book controller in the `admin` subdirectory would be declared like this:

```
class Admin::BookController < ActionController::Base
  # ...
end
```